

EVIDENCE

BOB-174 — Evidence pack

Revision: 1 **Last modified:** 2026-08-23T10:20:00Z **Guard:** tests/unit/api_layer/test_bob174_corrupt_hook_store.py (28 tests)

Change: download-proxy/src/api/hooks.py **Decision record:** DESIGN_DECISION.md beside this file

Host discipline throughout: GOMAXPROCS=2 nice -n 19 ionice -c 3, .venv/bin/python -m pytest -p no:schemathesis.
PYTEST_DISABLE_PLUGIN_AUTOLOAD deliberately NOT used (BOB-165).

1. RED — observed against the unmodified tree, before any source change

The guard was written first and run first. **15 failed, 13 passed.**

15 failed, 13 passed, 1 warning in 9.12s

```
FAILED ....:TestCorruptStoreIsNotReportedAsZeroHooks::test_get_does
FAILED ....:TestCorruptStoreIsNotReportedAsZeroHooks::test_get_does
FAILED ....:TestCorruptStoreIsNotReportedAsZeroHooks::test_get_does
FAILED ....:TestCorruptStoreIsNotReportedAsZeroHooks::test_error_de
FAILED ....:TestCreateRefusesRatherThanDestroying::test_create_is_i
FAILED ....:TestCreateRefusesRatherThanDestroying::test_create_is_i
FAILED ....:TestCreateRefusesRatherThanDestroying::test_create_is_i
FAILED ....:TestCreateRefusesRatherThanDestroying::test_the_operate
FAILED ....:TestCreateRefusesRatherThanDestroying::test_the_operate
FAILED ....:TestCreateRefusesRatherThanDestroying::test_recoverable
FAILED ....:TestDeleteRefusesRatherThanDestroying::test_delete_does
FAILED ....:TestWriteIsAtomic::test_a_crash_mid_write_leaves_the_pi
FAILED ....:TestWriteIsAtomic::test_the_surviving_store_is_still_p
FAILED ....:TestNonOSErrorWriteFailureIsPinned::test_a_serialisatio
```

The A2 data loss, reproduced verbatim

```
E AssertionError: 'prod-hook-0' is no longer in the hook store. A
truncated file is
E hand-repairable; an overwritten one is not. The create turned a
recoverable state
E into an unrecoverable one.
E assert 'prod-hook-0' in '[\n {\n   "hook_id":
"f71c5ebb-2d81-492b-87d8-61ed82e41549",
E   \n   "name": "bob174-new-hook", ... }\n]'
```

Three operator hooks in, one unrelated hook out. This is the item's reported chain, on the already-BOB-173-fixed tree.

The A5 corruption being manufactured, verbatim

```
E AssertionError: a write that FAILED PART-WAY left the store
truncated. The previous
E contents are gone and what remains is the exact corrupt input
that BOB-174's A1/A2
E chain then reads as 'no hooks' and overwrites.
E assert '[{"hook_id": "half-writ" == '[\n {\n   "hook_id":
"seeded-0", ...'
```

[{"hook_id": "half-writ — the fragment left by a failure mid-serialisation. That fragment is precisely the A1 input. The chain closes on itself, which is why the three defects are one item.

RED members that passed pre-fix, stated honestly (§11.4.6)

Two guards pass against the broken code and do **not** capture the escape by themselves. Neither is claimed to:

- `test_the_store_survives_a_refused_delete` — pre-fix, delete raised 404 *before* reaching the write, so the file was untouched for the wrong reason. It asserts the reality half that makes the 404 a lie;
`test_delete_does_not_report_not_found_for_an_unreadable_store` is the member that captured the defect.
 - `test_no_partial_or_temp_file_is_left_observable_after_a_failed_write` — pre-fix there were no temp files to leave. It pins a property of the new mechanism, not the old escape.
 - `test_dispatch_runs_no_hooks_on_a_corrupt_store` (added later, §5c) — pre-fix `_load_hooks` returned [], so no hooks were dispatched then either. It pins the **fail-closed** half of the dispatch decision against a future implementation that tries to salvage partial data; its partner
`test_dispatch_does_not_raise_on_a_corrupt_store` is the one that constrains the new code.
-

2. Two test bugs found and fixed during the RED→GREEN transition (§11.4.1)

Both were **instrument** defects, not product defects, and both are recorded because one of them was dangerous.

(a) An impossible assertion.

`test_recoverable_operator_data_is_still_recoverable` demanded all three hook names survive a 70%-truncated seed — but a truncation by definition cut `prod-hook-2` off, so the seed never contained it. No fix could deliver it. Corrected to assert on the names the seed *actually retains*.

(b) `monkeypatch.undo()` silently re-pointed the app at production state. The atomicity tests patched `json.dump`, then called `monkeypatch.undo()` to restore it mid-test. `monkeypatch` is a single function-scoped instance **shared with the harness fixture**, so `undo()` reverted *every* patch it had made — including `HOOKS_FILE`. The app was thereby re-pointed at the real `/config/download-proxy/hooks.json`, a GET read that non-existent path, and the test failed with `assert [] == ['prod-hook-0', ...]` for a reason having nothing to do with the code under test.

This is a §11.4.201(7)(c) instrument-path defect: **had the expected value been [], it would have passed vacuously while reading production state.** Replaced with a self-restoring one-shot patch, so no `undo()` is needed and the footgun is gone from the file. The reasoning is preserved in `_arm_one_shot_enospc`'s docstring so it cannot be reintroduced by a future edit.

3. GREEN

28 passed, 1 warning in 10.18s

All five hook-touching test files together, after reconciliation (§4):

```
tests/unit/api_layer/test_bob174_corrupt_hook_store.py
tests/unit/api_layer/test_bob173_hook_persistence_failure.py
tests/unit/api_layer/test_hooks_coverage.py
tests/unit/api_layer/test_hooks_endpoints.py
tests/unit/api_layer/test_hooks_remaining.py
--> 68 passed, 1 warning in 31.39s
```

`ruff check download-proxy/src/api/hooks.py tests/unit/api_layer/` →
All checks passed!

4. Two gates the fix legitimately broke — RECONCILED, not fake-passed (§11.4.120)

The full tests/unit/api_layer/ sweep surfaced 5 failures. Both causes were investigated before anything was touched; neither was a regression in the fix.

(a) test_hooks_coverage.py — two tests asserted the defect

```
fake_path.write_text("not json")
hooks = api.hooks._load_hooks()
assert hooks == []           # <- the defect, codified
```

These two tests are the reason the swallow looked deliberate. They assert the exact behaviour the item's acceptance criterion (a) removes. **Reconciled** to assert the new mechanism (pytest.raises(HookStoreCorruptError)), with the reasoning in-source. Note the sibling test_load_hooks_no_file was **left untouched and still passes** — that is the negative control proving MISSING still returns [].

(b) test_bob173_hook_persistence_failure.py — three DELETE tests, and a real behaviour change

BOB-173's seal_for_rewrite chmod'ed the hooks **file** to 0400 and probed with open(target, "a"). That was the correct needle while _save_hooks rewrote in place. The atomic write does not rewrite in place, so the seal stopped blocking anything and the delete write succeeded while the test expected a failure — the inverse of the §11.4.199 false-reproduction that file's own header warns about.

Measured before deciding (§11.4.6), not assumed:

```
append to a 0400 file : REFUSED (Permission denied)
os.replace over 0400   : SUCCEEDS
mkstemp in a 0500 dir : REFUSED (Permission denied)  <- the
atomic write fails here
```

This is a genuine, deliberate behaviour change and is recorded as one: a hooks.json chmod'ed 0400 no longer refuses a write, because os.replace relinks a directory entry and never consults the target's mode. It is accepted because (i) file mode on the app's own state store is not a documented protection mechanism anywhere in this project, (ii) theme_state.py already behaves this way, and (iii) the realistic deployment failure — the

EACCES-on-/config case BOB-135 actually hit — is a **directory** permission problem and is still caught, which is why BOB-173's *create* tests never failed.

Reconciled by sealing the DIRECTORY with the needle that matches the operation the code now performs (`_create_is_refused` — an atomic write must be able to create a new file). The resulting guard is a closer match to the code under test than the original was.

The premise is now pinned by a new test rather than left as a docstring claim:

```
TestAtomicWritePermissionSemanticsArePinned::test_a_read_only_target_file_does_not_stop_an_atomic_replace.
```

The reconciled gates still bite — verified, because a reconciled gate that stopped catching its own defect would be the §11.4.120 fake-pass in slow motion:

Mutation	Result
restore BOB-173's <code>_save_hooks</code> swallow	5 failed , 5 passed — including all 3 reconciled DELETE tests
restore the A1 <code>_load_hooks</code> swallow	2 failed , 10 passed — both reconciled coverage tests

5. Paired §1.1 mutations — every guard proven load-bearing

Each mutation restores one specific pre-fix behaviour; the source was restored from a pristine copy after each and the md5 re-verified (ea6dcb3441f946f10410d8309e6e8e5b) so no mutation residue could survive into the tree (§11.4.84).

#	Mutation	Result	Which guard bit
M1	restore <code>_load_hooks's</code> except <code>Exception: return []</code>	12 failed , 16 passed	A1 — GET, create-refusal, delete-404, data-survival
M2	restore <code>open(HOOKS_FILE, "w")</code> in place of the atomic write	3 failed , 25 passed	A5 — all three atomicity guards, and only those
M3	narrow <code>_save_hooks'</code>	2 failed , 26 passed	the previously-unpinned arm

#	Mutation	Result	Which guard bit
	except Exception → except OSError		(BOB-173 review finding)
M4	add archive_complete to VALID_EVENTS, absent from HookEventType	2 failed , 26 passed	A3 — the vocabulary pin
M5	drop create_hook's refusal, let the misread flow into the save	7 failed , 21 passed	A2 — the data-loss path specifically
M6	restore BOB-173's write-failure swallow	5 failed , 5 passed	the reconciled BOB-173 guard
M7	restore os.path.isfile in place of os.path.exists	1 failed , 28 passed	the non-file- at-store-path guard (§5b)
M8	let dispatch_event propagate the corrupt-store error	2 failed , 29 passed	the dispatch fail-closed guards

M2's narrowness is the useful signal: restoring the non-atomic write breaks **only** the atomicity tests and nothing else, which is what a correctly-scoped guard looks like.

M3 confirms the BOB-173 review's finding directly: before this change, except Exception → except OSError passed the entire suite. It now costs two tests.

5b. One defect found by self-review, not by the reported chain

`_load_hooks` guarded with `os.path.isfile`, which is **also False for a directory** (or fifo, socket, device node) at the store path. Probed rather than assumed:

```
os.path.isfile(dir) : False    -> treated as MISSING, GET says 0
hooks
os.path.exists(dir) : True
```

`open(dir)` raises `IsADirectoryError` (an `OSError` -> already handled)

So a directory at `hooks.json` was reported as “no hooks configured” — the *same* conflation the item is about, in a rarer flavour, and it would have survived a fix that kept `isfile`. Closed by changing one word to `os.path.exists` and letting it fall through to the `open`, where the existing `OSError` handler already covers it. A broken symlink stays `MISSING` under both spellings (`exists` follows links), which is correct — nothing readable is there.

Pinned by

`test_a_directory_at_the_store_path_is_not_reported_as_zero_hooks`, and M7 confirms it is load-bearing.

5c. A decision that was recorded but not guarded

`dispatch_event`’s corrupt-store behaviour (§3 of the decision record — log, dispatch nothing, do not propagate) was **decided and written down but initially left untested**. A recorded decision with no guard is one edit away from silently reverting, and the reverting edit is an attractive one: “why is this swallowing an exception?” is exactly what a future reader will ask. Closed by `TestDispatchFailsClosedWithoutBreakingUnrelatedRequests`, which asserts **both** halves — it does not raise (so a corrupt hooks file cannot 500 an unrelated search or download) **and** it runs no hooks (so the side effect fails closed). Each half alone is satisfiable by a wrong implementation, which is why both are asserted. M8 confirms they bite.

6. Negative controls (§11.4.201(1)) — the fix does not fail closed by failing always

Control	Assertion
<code>test_get_returns_an_empty_list_when_no_store_exists</code>	MISSING file → 200 { "hooks": [], "count": 0 }
<code>test_create_works_on_a_fresh_install</code>	MISSING file → create succeeds and persists

Control	Assertion
test_an_empty_json_list_is_a_healthy_store_not_a_corrupt_one	[] on disk stays healthy (what deleting the last hook leaves)
test_get_lists_the_configured_hooks	VALID file → all three listed
test_create_appends_and_preserves_the_existing_hooks	VALID file → appends, does not replace
test_delete_removes_only_the_named_hook	VALID file → removes exactly one
test_delete_of_an_absent_hook_still_404s_on_a_valid_store	the ordinary not-found path is not swallowed by the corrupt-store guard
test_load_hooks_no_file (pre-existing, untouched)	the MISSING-is-not-CORRUPT boundary, asserted at the unit layer

The missing-file controls are the sharp ones: [] was the **correct** answer for that state, and a fix that collapsed it into the corrupt case would 500 every fresh install's hooks tab and refuse every create — worse than the defect being fixed.

7. Atomicity proof — not merely “it still writes”

TestWriteIsAtomic drives a **faithful** failure rather than a synthetic one: `json.dump` writes a partial fragment and then raises `OSError(ENOSPC)`, modelling a full disk or a kill mid-serialisation at the point that matters. Four assertions:

1. the previous store is **byte-identical** afterwards (the atomicity proof — with an in-place open the original is destroyed at open time, before the failure even occurs);
2. the store is **still parseable and still lists the original hooks** through the real API afterwards;
3. **no partial or temp file is observable** after a failed write;
4. **no temp file is left** after a successful write.

M2 confirms (1) and (2) are the load-bearing pair.

8. Honest boundaries

- **The dashboard still shows “No hooks registered” on a corrupt store.** The API is now honest; the UI is not. Fixing it needs an error callback in `loadHooks()` and a template branch, both in frontend/ — outside this item’s scope and live under a sibling stream. Recorded in `DESIGN_DECISION.md` §1 as a known gap and the natural follow-up.
- **dispatch_event’s corrupt-store arm is log-only.** Accepted only there, because that coroutine is awaited inline by the search and download handlers and raising would 500 unrelated capabilities. Reasoning and the conditions under which it should change: `DESIGN_DECISION.md` §3.
- **Not verified against a real ENOSPC or a real SIGKILL.** The injection models the failure faithfully but is not the syscall-level event.
- **No live-container run.** Everything here is the real FastAPI app driven through `TestClient` over real files on real temp paths — no mocks of the code under test — but not a running qbittorrent-proxy container.
- **A corrupt store is not now impossible.** The atomic write closes the crash-mid-write vector this code owns; a bad hand-edit or filesystem damage still produces one, which is why the read guard exists too.
- **DESIGN_DECISION.{html,pdf,docx} are STALE.** An export pipeline generated them at 11:50 from an earlier revision of the `.md`, which I then edited at 12:00 (adding the permission-semantics and non-file-path decisions). I did not regenerate them: the exporter (`scripts/generate_markdown_exports.sh`) is itself modified by a sibling stream right now, and running it would

sweep files outside this item. Flagged rather than silently left divergent (§11.4.106) — the .md is authoritative.

- **_save_hooks is still not concurrency-safe against a second writer.** `os.replace` makes each write atomic, so no reader ever sees a partial file — a real improvement — but two concurrent create requests can still interleave load→append→save and lose one hook. That is a separate defect (last-writer-wins), out of scope here, and **not** claimed to be fixed by this change.